Nonlinear Model Predictive Control using Decoupled A-B Net Formulation for Carbon Capture Systems – Comparison with Algorithmic Differentiation Approach*

Priyadarshi Mahapatra, Jinliang Ma and Stephen E. Zitney, *National Energy Technology Laboratory, USA*

***Abstract*— A novel state-space-based multivariable constrained nonlinear model predictive control (NMPC) strategy is described. The NMPC strategy uses enhanced Decoupled A-B Net (DABNet) based dynamic reduced models (D-RMs) as core predictive models. The control objective cost-function and its analytical derivatives, explicitly utilize the decoupled nature of the DABNet model, which provides computational advantages over automatic differentiation (AD) methodologies. The output-node parallelization strategy described in this paper is demonstrated to provide comparable computational advantages as that found in linear multi-rate MPC algorithms, while furnishing superior disturbance rejection and setpoint tracking characteristics of the NMPC. The approach has been presented and compared on a multi-time-scale solid-sorbent-based bubbling fluidized bed (BFB) carbon capture process.**

I. Introduction

Model-based controllers can achieve superior performance in comparison to model-free controllers such as PID controllers. However, the computational cost for model-based controllers can be prohibitive for many industrial applications. Nonlinear model-based control techniques such as nonlinear model predictive control (NMPC) can provide significantly better control performance than their linear counterparts, but are difficult to implement due to the need for accurate yet computationally efficient predictive models of the corresponding process models.

The core predictive-model identification methodology used in this study is based on Decoupled A-B Net (DABNet), first proposed in [1]. This methodology can approximate nonlinear, causal, discrete time-invariant, multiple-input single-output (MISO) systems with fading memory. The original DABNet algorithm was significantly improved and extended for incorporation into the D-RM Builder software tool, which was developed under the auspices of the U.S. Department of Energy's Carbon Capture Simulation Initiative (CCSI). The improved methodology in D-RM Builder, as detailed in [2], can accurately and efficiently capture the dynamics of multiple-input multiple-output (MIMO) systems with multi-time scale transients. It also includes advanced sampling, training and validation techniques.

This paper discusses the development and application of a specific NMPC formulation, DABNet-based NMPC, which was developed as part of the Advanced Process Control (APC) Framework software given in [3]. Similar to D-RM Builder, the APC Framework was developed under CCSI [4]. In particular, this paper describes the computational benefits of optimizing the underlying nonlinear control objective by exploiting the decoupled nature of the prediction-model.

II. DABNet-based NMPC Formulation

A. DABNet Model

The model consists of a discretized linear dynamic mapping or marching function of state-space vector represented by discrete Laguerre functions [1], followed by a nonlinear static mapping from state-space vectors to outputs. The inputs (u) are characterized by all manipulated variables and measured disturbances, which are deemed necessary for the controller. Similarly, the outputs (y) are all the measured process variables (also referred to as control variables, CV) which need to be actively controlled or aid in quantifying model uncertainty through state/disturbance estimation. Being a data-driven or black-box type D-RM, any process or plant data, not incorporated within the list of inputs/outputs cannot be utilized in the control implementation.

For a system with multiple outputs, multiple sets of DABNet models are used to describe the system, one DABNet for each output variable. The linear dynamic system is initially spanned by a set of discrete Laguerre systems and then cascaded with an artificial neural network (ANN) with a single hidden layer of neurons. The original Laguerre dynamic system is then reduced through linear balancing as a part of the identification process, which reduces the dimensionality of the state space and hence the number of inputs to the ANN. The balanced linear dynamic system is cascaded with a new ANN and trained again. The balanced nonlinear dynamic model of DABNet is shown in (1) and (2)

$$\mathbf{x}_{ij}(k+1) = \mathbf{A}_{ij}\mathbf{x}_{ij}(k) + \mathbf{B}_{ij}u_j(k) \tag{1}$$

$$y_i(k+1) = NN^i\left(\mathbf{x}_i(k+1)\right) \tag{2}$$

where $\mathbf{x}_{ij}$, $\mathbf{A}_{ij}$, and $\mathbf{B}_{ij}$ are the decoupled state space vector and time-invariant state-space coefficient matrices, respectively, for input-output pair *j*-*i*. The index *k* is the

*Research supported by U.S. Department of Energy, Office of Fossil Energy as part of Carbon Capture Simulation Initiative (CCSI).

P. Mahapatra is with the National Energy Technology Laboratory, Pittsburgh, PA 15236 USA (phone: 412-386-5044; fax: 412-386-4604; e-mail: priyadarshi.mahapatra@netl.doe.gov).

J. Ma is with the National Energy Technology Laboratory, Morgantown, WV 26507 USA (e-mail: jinliang.ma@netl.doe.gov).

S. E. Zitney is with the National Energy Technology Laboratory Morgantown, WV 26507 USA (e-mail: steve.zitney@netl.doe.gov).

discrete time index and u_j and y_i are j^{th} input and i^{th} output respectively. The state, $\mathbf{x}_i$, is the augmented state-space vector for i^{th} output as shown in (3) and comprises n_u decoupled state-space vectors, one for each input. NN^i represents an ANN function for the static mapping from the augmented state-space vector to the output.

$$\mathbf{x}_i = \begin{bmatrix} \mathbf{x}_{i1} & \mathbf{x}_{i2} & \cdots & \mathbf{x}_{i,nu} \end{bmatrix}^T \tag{3}$$

A detailed description of the DABNet-based D-RM system-identification process is given in [2]. The D-RMs are trained and validated using two different sets of transient data. The methodology involves the generation of step-response data from high-fidelity plant simulations using Latin Hypercube Sampling (LHS). Based on heuristic knowledge of the process, an initial estimate for process and dead-times are provided. The initial decoupled state-space matrices and states are cast in an augmented-form as shown in (2) and (3). The weights inside the ANN are then trained to match the actual response (or, as in this case, match data from the high-fidelity model). The augmented state-space coefficient matrices are reduced by balanced realization as outlined in [1]. Using the balanced dynamic model, the new ANN function in (2) is re-trained.

B. Model Prediction and Derivative Formulation

In any MPC formulation, the model-output predictions are needed for finite time-steps into the future. This is the prediction horizon, *P*. Given a sequence of input values over *P* time-steps, denoted by $\mathbf{u}_k$, $\mathbf{u}_{k+1}$, $\mathbf{u}_{k+2}$,…, $\mathbf{u}_{k+P-1}$, the state-space vector for each output, mathematically evolves as shown in (4).

$$\begin{bmatrix} \mathbf{x}^i_{k+1} \\ \mathbf{x}^i_{k+2} \\ \mathbf{x}^i_{k+3} \\ \vdots \\ \mathbf{x}^i_{k+P} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_i \\ \mathbf{A}_i^2 \\ \mathbf{A}_i^3 \\ \vdots \\ \mathbf{A}_i^P \end{bmatrix} \mathbf{x}^i_k + \begin{bmatrix} \mathbf{B}_i & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{A}_i\mathbf{B}_i & \mathbf{B}_i & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{A}_i^2\mathbf{B}_i & \mathbf{A}_i\mathbf{B}_i & \mathbf{B}_i & & \mathbf{0} \\ \vdots & \vdots & \vdots & & \vdots \\ \mathbf{A}_i^{P-1}\mathbf{B}_i & \mathbf{A}_i^{P-2}\mathbf{B}_i & \mathbf{A}_i^{P-3}\mathbf{B}_i & \cdots & \mathbf{B}_i \end{bmatrix} \begin{bmatrix} \mathbf{u}_k \\ \mathbf{u}_{k+1} \\ \mathbf{u}_{k+2} \\ \vdots \\ \mathbf{u}_{k+P-1} \end{bmatrix} \tag{4}$$

Note the modified nomenclature $\mathbf{x}^i_k \equiv \mathbf{x}_i(k)$ for augmented state-space at time-step *k* corresponding to output *i*. The model predictions are thereafter represented in terms of M (control-horizon) input-changes, $\Delta\mathbf{u}_k$, $\Delta\mathbf{u}_{k+1}$, $\Delta\mathbf{u}_{k+2}$,…, $\Delta\mathbf{u}_{k+M-1}$, instead of the actual input values.

The predictions for i^{th} scalar output, with the simplified nomenclature similar to state variables, are given in (5).

$$Y^i_P \equiv \begin{bmatrix} y^i_{k+1} \\ y^i_{k+2} \\ y^i_{k+3} \\ \vdots \\ y^i_{k+P} \end{bmatrix} = \begin{bmatrix} NN^i(\mathbf{x}^i_{k+1}) \\ NN^i(\mathbf{x}^i_{k+2}) \\ NN^i(\mathbf{x}^i_{k+3}) \\ \vdots \\ NN^i(\mathbf{x}^i_{k+P}) \end{bmatrix} \tag{5}$$

The deterministic structure of DABNet-based black-box models can be used to evaluate analytical derivatives with relative ease. If coded efficiently, the derivative evaluations may be orders of magnitude faster, are exact in terms of accuracy, and require significantly less memory, compared to other numeric-differentiation and derivative-free methods. The first-derivative of states (corresponding to output *i*) over prediction horizon, *P*, with respect to input-changes over control horizon, *M*, is represented by $\frac{\partial \mathbf{X}^i_P}{\partial \mathbf{\Delta U}_M}$, and is obtained using (4). The first-derivative of outputs with respect to states, over *P* corresponding to output *i*, is given by (6).

$$\frac{\partial Y^i_P}{\partial \mathbf{X}^i_P} = blkdiag\left[\left.\frac{\partial NN^i}{\partial \mathbf{x}^i}\right|_{\mathbf{x}^i_{k+1}} \quad \cdots \quad \left.\frac{\partial NN^i}{\partial \mathbf{x}^i}\right|_{\mathbf{x}^i_{k+P}} \right] \tag{6}$$

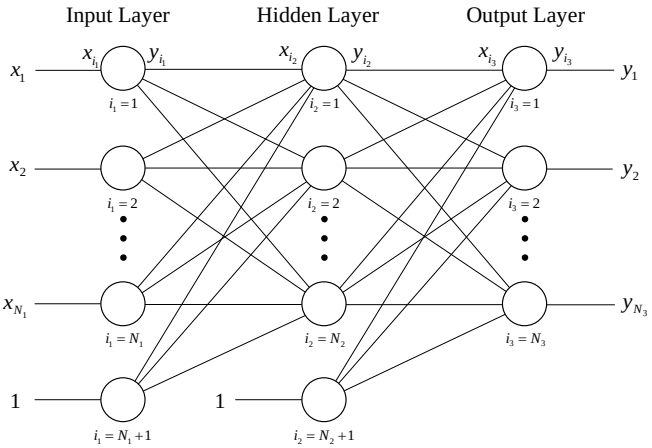


Figure 1. Representation of a feedforward neural network with single hidden layer

The first-derivatives of neural network output w.r.t. state inputs $\left.\frac{\partial NN^i}{\partial \mathbf{x}^i}\right|_{\mathbf{x}^i}$ are obtained utilizing chain rules corresponding to activation function derivatives for each neuron. Fig. 1 shows a feedforward neural network with a single hidden layer as used in the DABNet model.

The first derivative of each output over entire prediction horizon with respect to changes over entire control horizon is given by (7)

$$\frac{\partial Y^i_P}{\partial \mathbf{\Delta U}_M} = \frac{\partial Y^i_P}{\partial \mathbf{X}^i_P} \frac{\partial \mathbf{X}^i_P}{\partial \mathbf{\Delta U}_M} \tag{7}$$

The corresponding derivative of the pth prediction step w.r.t. the mth control move is represented by $\frac{\partial y_p^i}{\partial \Delta \mathbf{u}_m} \equiv \frac{\partial Y_P^i}{\partial \Delta \mathbf{U}_M}\{p,m\}$. This is given by (8), for $m \le p$. For other combinations of *p* and *m*, the terms are zero block matrices.

$$\frac{\partial y_p^i}{\partial \Delta \mathbf{u}_m} = \frac{\partial NN^i}{\partial \mathbf{x}^i}\bigg|_{\mathbf{x}_{k+p}^i} (\mathbf{A}_i^{p-m}\mathbf{B}_i + ... + \mathbf{B}_i) \tag{8}$$

The second-derivative is much more complex and is dealt with on a per-output, per-prediction-step basis (y_p^i). This is given in (9).

$$\frac{\partial^2 y_p^i}{\partial (\Delta \mathbf{U}_M)^2} = \begin{bmatrix} \frac{\partial^2 y_p^i}{\partial \Delta \mathbf{u}_1 \partial \Delta \mathbf{u}_1} & \cdots & \frac{\partial^2 y_p^i}{\partial \Delta \mathbf{u}_M \partial \Delta \mathbf{u}_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 y_p^i}{\partial \Delta \mathbf{u}_1 \partial \Delta \mathbf{u}_M} & \cdots & \frac{\partial^2 y_p^i}{\partial \Delta \mathbf{u}_M \partial \Delta \mathbf{u}_M} \end{bmatrix} \tag{9}$$

Each individual term in the Hessian denotes the second-derivative of y_p^i w.r.t. the m_1^{th} and m_2^{th} control move and is given by $\frac{\partial^2 y_p^i}{\partial (\Delta \mathbf{U}_M)^2}\{m_1, m_2\} \equiv \frac{\partial^2 y_p^i}{\partial \Delta \mathbf{u}_{m_2} \partial \Delta \mathbf{u}_{m_1}}$. This is given by (10). The second-derivatives of the neural network output w.r.t. state input $\frac{\partial^2 NN^i}{\partial (\mathbf{x}^i)^2}\bigg|_{\mathbf{x}^i}$ can be calculated by the chain rule as earlier.

$$\frac{\partial^2 y_p^i}{\partial \Delta \mathbf{u}_{m_2} \partial \Delta \mathbf{u}_{m_1}} = (\mathbf{A}^{p-m_1}\mathbf{B} + ... + \mathbf{B})_i^T \frac{\partial^2 NN^i}{\partial (\mathbf{x}^i)^2}\bigg|_{\mathbf{x}_{k+p}^i} (\mathbf{A}^{p-m_2}\mathbf{B} + ... + \mathbf{B})_i \tag{10}$$

C. Output-Node Parallelization

Due to decoupled nature of DAB-Net model architecture, each output and its corresponding derivatives, i.e., Y_P^i, $\frac{\partial Y_p^i}{\partial \Delta \mathbf{U}_M}$ and $\frac{\partial^2 y_p^i}{\partial (\Delta \mathbf{U}_M)^2}$, may be computed by multiple nodes or threads for parallel evaluation. On a multi-core CPU enabled with multithreading capability, and appropriate software tools capable of distributing computational workload, this deconvolution can add significant speed as opposed to sequential approach on a single thread. The benefits of output-node parallelization is more pronounced when using optimization algorithms that require exact Hessian evaluation such as that in IPOPT [5].

In current study, MATLAB's Parallel Computing Toolbox was utilized to distribute individual threads corresponding to each output-node *i*.

D. Nonlinear Model Predictive Control

Zhao et al. [6] had used the original DABNet-based black-box model as proposed in [1], in an industrial NMPC framework which uses an integrated partial least-squares and neural net model. However, they use a multi-step Newton-type algorithm for solving the nonlinear programming (NLP) problem which uses a linearized approximation of the system around the nominal trajectory. They also handled output constraints as "soft-constraints" using the l_1 penalty treatment. The current approach utilizes a more sophisticated interior-point-based constrained NLP optimizer. To account for plant-model mismatch and unmeasured disturbances, a simple "additive output disturbance" similar to a dynamic matrix control scheme is incorporated. The APC Framework is capable of using explicit Kalman filters, such as extended and unscented Kalman filters, for handling plant-model mismatch and unmeasured disturbances, which has not been discussed here.

The control objective function is given in (11), where a desired setpoint trajectory $\mathbf{R}_P = \begin{bmatrix} \mathbf{r}_{k+1} & \mathbf{r}_{k+2} & \dots & \mathbf{r}_{k+P} \end{bmatrix}^T$, over a given prediction horizon is specified.

$$\min_{\Delta \mathbf{u}_k ... \Delta \mathbf{u}_{k+M-1}} J = \sum_{i=1}^{P} (\mathbf{r}_{k+i} - \mathbf{y}_{k+i})^T \mathbf{w}_y (\mathbf{r}_{k+i} - \mathbf{y}_{k+i}) + \sum_{j=0}^{M-1} \Delta \mathbf{u}_{k+j}^T \mathbf{w}_u \Delta \mathbf{u}_{k+j} \tag{11}$$

Here, $\mathbf{w}_y$ represents the output weighting matrix, while $\mathbf{w}_u$ represents the input weighting matrix. This formulation closely corresponds to the NMPC formulation used in industry.

The analytical gradient (and Hessian) evaluation of the cost function is necessary for significant speedup when used in conjunction with many NLP optimizers. The cost gradient vector, represented by $\frac{\partial J}{\partial \Delta \mathbf{U}_M}$, is evaluated as shown in (12) and (13).

$$\frac{\partial J}{\partial \Delta \mathbf{U}_M} = \begin{bmatrix} \frac{\partial J}{\partial \Delta \mathbf{u}_1} & \frac{\partial J}{\partial \Delta \mathbf{u}_2} & \dots & \frac{\partial J}{\partial \Delta \mathbf{u}_M} \end{bmatrix} \tag{12}$$

$$\frac{\partial J}{\partial \Delta \mathbf{u}_m} = \sum_{p=1}^{P} \left(-(\mathbf{r}_p - \mathbf{y}_p)^T (\mathbf{w}_y + \mathbf{w}_y^T) \frac{\partial \mathbf{y}_p}{\partial \Delta \mathbf{u}_m} \right) + \Delta \mathbf{u}_m^T (\mathbf{w}_u + \mathbf{w}_u^T) \tag{13}$$

The desegregation of individual outputs in the DABNet formulation also provides unique benefits in the formulation steps for generating Hessian matrices by avoiding vector derivatives of matrices or tensor variables, significantly reducing computational cost.

$$\frac{\partial^2 J}{\partial(\Delta\mathbf{U}_M)^2}=\begin{bmatrix} \frac{\partial^2 J}{\partial\Delta\mathbf{u}_1\partial\Delta\mathbf{u}_1} & \cdots & \frac{\partial^2 J}{\partial\Delta\mathbf{u}_M\partial\Delta\mathbf{u}_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 J}{\partial\Delta\mathbf{u}_1\partial\Delta\mathbf{u}_M} & \cdots & \frac{\partial^2 J}{\partial\Delta\mathbf{u}_M\partial\Delta\mathbf{u}_M} \end{bmatrix} \quad (14)$$

Each individual block matrix in the Hessian denotes the second derivative of J w.r.t. m_1^{th} and m_2^{th} control move and is defined as $\frac{\partial^2 J}{\partial(\Delta\mathbf{U}_M)^2}\{m_1,m_2\}\equiv\frac{\partial^2 J}{\partial\Delta\mathbf{u}_{m_2}\partial\Delta\mathbf{u}_{m_1}}$, which is given by (15).

$$\frac{\partial^2 J}{\partial\Delta\mathbf{u}_{m_2}\partial\Delta\mathbf{u}_{m_1}}=\zeta(\mathbf{w}_u+\mathbf{w}_u^T)+$$
$$\sum_{p=1}^{P}\left(\begin{array}{l}\left(\frac{\partial\mathbf{y}_p}{\partial\Delta\mathbf{u}_{m_1}}\right)^T\left(\mathbf{w}_y+\mathbf{w}_y^T\right)\frac{\partial\mathbf{y}_p}{\partial\Delta\mathbf{u}_{m_2}}- \\ \sum_{i=1}^{ny}\left(\left(\mathbf{r}_p-\mathbf{y}_p\right)^T\left(\mathbf{w}_y+\mathbf{w}_y^T\right)\right)_i\frac{\partial^2 y_p^i}{\partial\Delta\mathbf{u}_{m_2}\partial\Delta\mathbf{u}_{m_1}}\end{array}\right) \quad (15)$$

where, $\zeta=1$ when $m_1=m_2$ and 0 otherwise. First-derivative terms are evaluated in (8) and second-derivative terms are given in (10).

Direct use of the cost-function Hessian is rarely found within constrained interior-point-based NLP solvers. An unconstrained and trust-region-reflective-based NLP approach may use the above Hessian directly within its problem specification.

Due to space restrictions, detailed constraint formulation is not discussed in this text. It must be noted that Hessians of nonlinear inequality functions are incorporated within the second-derivative of the Lagrangian function. This is utilized by many interior-point-based constrained NLP solvers/optimizers including *fmincon*, in MATLAB's Optimization Toolbox [7], and IPOPT [5].

E. Automatic Differentiation Formulation

In recent years, breakthroughs in automatic evaluation of derivatives have made implementation of NMPC significantly more feasible. This methodology, also known as automatic differentiation (AD) or algorithmic differentiation [8][9], automatically augments computer function evaluations with statements for computation of derivatives. A major argument against AD is the removal of the programmer from the code. In performance-critical applications, such as many simulations systems, having hard-coded, optimizable code for calculating derivatives can significantly reduce computational time. For example, modifying the calculation of derivatives and parallelizing parts of the sequence, a significant speed increase in simulation systems is possible. Since AD algorithms may not allow the programmer to optimize the code calculating derivatives, the evaluation may incur higher computational cost in certain scenarios, especially when higher-order derivatives are required. In addition, third party software dependencies may render the practical implementation of such an evaluation useless.

This study relies heavily on exact-analytical evaluation of gradients and Hessians to aid NLP solvers for control calculation, as described previously. However, the efforts involved in programming such evaluation routines may be negated if the computational benefits of this approach do not surpass those obtained using AD algorithms. To quantify this, an AD methodology was formulated for evaluating first and second derivatives of the control objective function. To this end, the approach uses the ADiMAT MATLAB toolbox [10] featuring a hybrid source transformation and object-oriented programming approach for MATLAB codes [11]. This AD-tool generates code for first and second-order derivative computation using the forward mode of AD.

III. NMPC OF BFB ADSORBER-REACTOR

For this control study, a DABNet model of a single bubbling fluidized bed (BFB) absorber-reactor is developed using the D-RM Builder tool and NMPC is implemented within the APC Framework. The BFB adsorber uses a mesoporous silica-supported polyethylenemine-impregnated solid sorbent to capture CO_2 from flue gas exiting a power plant. The adsorber system consists of two BFB vessels in countercurrent series with the flue-gas entering the bottom of the lower bed and sorbent entering the top of the top bed. Each bed has internal cooling tubes for heat removal. A detailed first-principles-based steady-state model of the BFB adsorber, using Aspen Custom Modeler (ACM) [12], was presented in [13]. This was extended to a detailed dynamic model in [14]. The preliminary D-RM development and control formulation, including various software interfaces, using CCSI's D-RM Builder and APC Framework tools was described in [3]. NMPC comparison utilizing first-principles-based reduced-order model vs. current data-driven approach was conducted in [15]. The presence of multiple time-scales in this process due to two-phase, gas-solid reaction dynamics, requiring the use of "two-poles" is discussed in [2]. This multiple time-scale effect renders the BFB control problem very challenging due to nonlinearities involved in the process.

A process and instrumentation diagram (P&ID) of the CO_2 capture process used in this case-study is shown in Fig. 2. This is a 3-input 1-output system consisting of sorbent flowrate ($F_{sorbent}$) as the manipulated variable (MV), flue-gas flowrate ($F_{fluegas}$) and CO_2 composition (Z_{CO2}) as the two disturbance variables (DV) and CO_2 capture percentage (C_{CO2}) as the controlled variable (CV).

The process consists of various lower-level controllers including level control of the downcomers, adsorber bed temperature control, and sorbent flow control. The D-RM is generated from the rigorous dynamic ACM model with the lower-level control in place. A sampling-time of 50 sec is used. The D-RM is generated based on ±10% of the normal operating conditions [14]. To generate the step change sequence for training, four sets of LHS samples are generated. It took approximately 300 sec of CPU time to train the DABNet model and optimize the pole values on a 2.2 GHz 8-core Intel processor. Fig. 3 compares the ACM-based

high-fidelity "plant" model responses of the validation sequence with the D-RM results. The average error when training the ANN of the balanced model is 1.09×10^{-4} and the R^2 value for the validation data is 0.998, indicating a very accurate D-RM. For comparison, the linear state-space model identified using MATLAB system identification toolbox is also shown. The linear model is third order and used sub-space identification method.

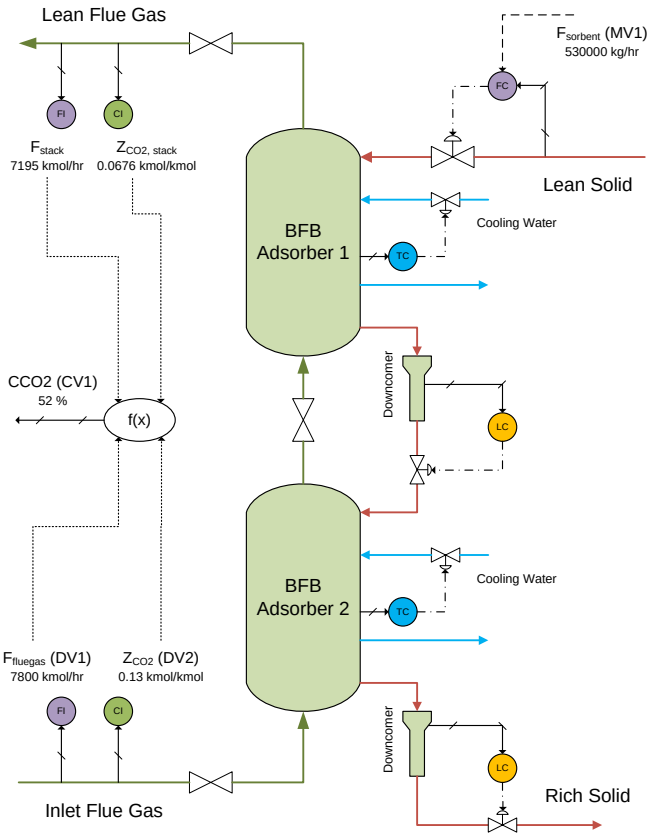


Figure 2. Two-stage BFB adsorber process and instrumentation diagram showing various inventory-layered controllers and supervisory control I/O variables along-with nominal values

The control objective is to maintain the setpoint for the CO_2 removal fraction and reject disturbances by manipulating the inlet flowrate of the solid sorbent. After large disturbances occur, the long-term average CO_2 capture percentage may deviate significantly from the desired capture percentage. Therefore, it may be necessary to change the target setpoint to achieve the average CO_2 removal fraction over a specified period of time. This will require the controller to also exhibit good performance with setpoint tracking, in addition to having good performance with disturbance rejection.

The NMPC formulation is then compared with linear MPC to evaluate the control performance and the computational time for optimization performed at each time interval. Each MPC formulation uses the same objective function, additive output disturbance (AOD) as feedback mode and constraint formulation. IPOPT is used as NLP solver for the NMPC formulation while MATLAB's quadratic programming is used for linear MPC. Both use similar tolerance specification for optimizer convergence. A sampling time of 50 sec, prediction horizon of 8 (400 sec), control horizon of 8 (400 sec) and W_u/W_y of 10^{-11} has been used in all the investigations. Similar CPU hardware has been utilized for comparing and contrasting different formulations of this case-study.

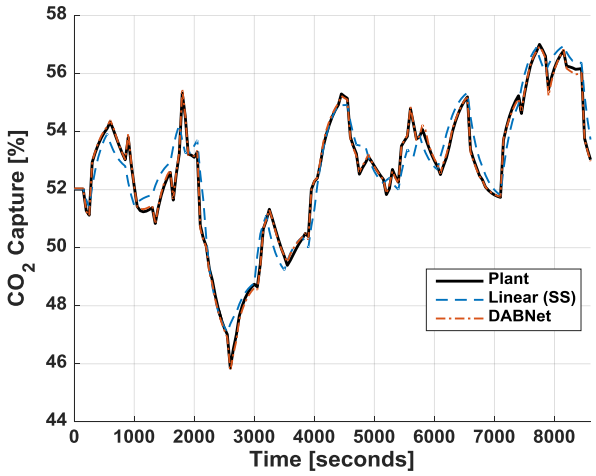


Figure 3. System identification (validation) plots showing model predictions vs. high fidelity "plant" measurements for LHS-based step-input sequence of all three input variables

Fig. 4 shows the controller responses to large fluctuations in inlet flue-gas, both flowrate (load) and composition. It can be seen that the DABNet-based NMPC provides superior control performance compared to conventional MPC, with a faster settling time and significantly less overshoot. This is more prominent at off-design conditions (60% capture). The Mean Integral Square Error (MISE) [15] is 0.953 for linear MPC whereas it is 0.826 for NMPC. Fig. 5 presents comparison of the computational expense between traditional MPC formulation, DABNet-based NMPC and NMPC utilizing Automatic Differentiation method. It may be seen that the AD methodology is significantly more computationally expensive compared to exact derivative formulation using output-node parallelization.

IV. Conclusion

In conclusion, the primary focus of this work was to establish superior computational advantage of the DABNet based NMPC described in this study utilizing exact analytical derivative over control optimization methodology using generic AD tools. The theoretical basis for the NMPC approach and utilization of the decoupled nature of underlying prediction-model using output-node parallelization has been detailed. The control-performance comparison was demonstrated on a complex CO_2 capture BFB adsorber process model.

Acknowledgment

The authors thank Benjamin P. Omell for developing BFB adsorber dynamic-model and David C. Miller for leading the Carbon Capture Simulation Initiative (CCSI), providing critical insights and direction to the APC Framework Toolset development within CCSI and supporting this research.

Disclaimer: *This project was funded by the Department of Energy, National Energy Technology Laboratory, an agency of the United States Government, through a support contract with AECOM. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.*

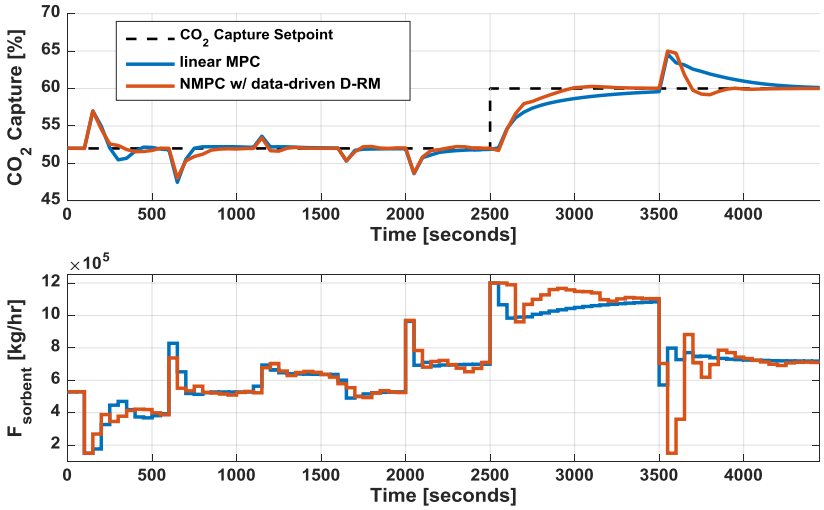


Figure 4. Comparison of control performance for the linear MPC and NMPC with DAB-Net formulation.

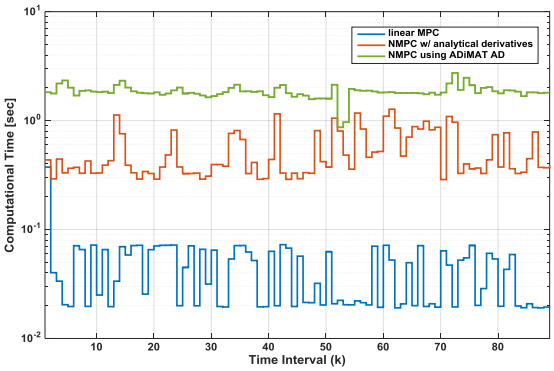


Figure 5. Computational cost at each time interval for LMPC, DABNet-NMPC with Analytical Derivative and DABNet-NMPC utilizing ADiMAT Automatic Differentiation

REFERENCES

[1] G. B. Sentoni, L. T. Biegler, J. B. Guiver, and H. Zhao, "State-space nonlinear process modeling: identification and universality," *AIChE Journal*, vol. 44, no. 10, pp. 2229-2239, 1998.

[2] J. Ma, P. Mahapatra, S. E. Zitney, L. T. Biegler, and D. C. Miller, "D-RM Builder: A software tool for generating fast and accurate nonlinear dynamic reduced models from high-fidelity models," *Computers & Chemical Engineering*, vol. 94, pp. 60-74, 2016.

[3] P. Mahapatra, J. Ma, B. Ng, D. Bhattacharyya, S. E. Zitney, and D. C. Miller, "Integrated Dynamic Modeling and Advanced Process Control of Carbon Capture Systems," *Energy Procedia*, vol. 63, pp. 1354-1367, 2014.

[4] D.C. Miller, D. Agarwal, D. Bhattacharyya, J. Boverhof, Y. Chen, J. Eslick, J. Leek, J. Ma, P. Mahapatra, B. Ng, and N.V. Sahinidis, "Innovative Computational Tools and Models for the Design, Optimization and Control of Carbon Capture Processes." in *Process Systems and Materials for CO_2 Capture: Modelling, Design, Control and Integration*, A.I. Papadopoulos and P. Seferlis, Ed., Wiley, 2017, chap.12, p.311.

[5] A. Wächter, and L. T. Biegler, "On the Implementation of a Primal-Dual Interior Point Filter Line Search Algorithm for Large-Scale Nonlinear Programming", *Mathematical Programming*, vol. 106, no. 1, 25-57, 2006.

[6] H. Zhao, J. Guiver, R. Neelakantan, and L. T. Biegler, "A nonlinear industrial model predictive controller using integrated PLS and neural net state-space model," *Control Engineering Practice*, vol. 9, no. 2, pp. 125-133, 2001.

[7] Mathworks, Inc., "Optimization Toolbox," MATLAB documentation, available at http://www.mathworks.com/products/optimization/, 2017a.

[8] M. Bartholomew-Biggs, S. Brown, B. Christianson, and L. Dixon, "Automatic differentiation of algorithms," *Journal of Computational and Applied Mathematics*, vol. 124, no. 1, pp. 171-190, 2000.

[9] Y. Cao and R. Al-Seyab, "Nonlinear model predictive control using automatic differentiation," In *European Control Conference (ECC 2003)*, Cambridge, UK, 2003.

[10] ADiMAT MATLAB Toolbox, in http://www.adimat.de, accessed March 2017.

[11] C. H. Bischof, H. M. Bücker, B. Lang, A. Rasch, and A. Vehreschild, "Combining Source Transformation and Operator Overloading Techniques to Compute Derivatives for MATLAB Programs," In *Proceedings of the Second IEEE International Workshop on Source Code Analysis and Manipulation (SCAM 2002)*, IEEE Computer Society, 2002.

[12] AspenTech®, Aspen Custom Modeler, in http://www.aspentech.com/products/engineering/aspen-custom-modeler, 2016.

[13] A. Lee and D. C. Miller, "A one-dimensional (1-D) three-region model for a bubbling fluidized-bed adsorber," *Industrial & Engineering Chemistry Research*, vol. 52, no. 1, pp. 469-484, 2013.

[14] S. Modekurti, D. Bhattacharyya, and S. E. Zitney, "Dynamic Modeling and Control Studies of a Two-Stage Bubbling Fluidized Bed Adsorber-Reactor for Solid-Sorbent CO_2 Capture," *Industrial & Engineering Chemistry Research*, vol. 52, pp. 10250-10260, 2013.

[15] B. P. Omell, J. Ma, P. Mahapatra, M. Yu, A. Lee, D. Bhattacharyya, S. E. Zitney, and D. C. Miller, "Advanced Modeling and Control of a Solid Sorbent-Based CO_2 Capture Process," *IFAC-PapersOnLine*, vol. 49, no. 7, pp. 633-638, 2016.